scatter/linear

An AgentMPI collective scaling analysis

Abstract

The linear scatter is the flat fan-out: the root sends `payloads[d]` directly to each rank d, and every other rank performs one receive. It costs $p-1$ messages and $(p-1)n$ tokens — the minimum any scatter can achieve, since every rank's work unit must cross the network exactly once — and the sweep confirms both at all twenty-one measured sizes, 7 at $p=8$ and 31 at $p=32$. It is the dual of the linear gather and it inherits the same asymmetry from the opposite direction: at $p=32$ exactly one rank sends anything, and it sends thirty-one times. The closed form prices that at $p-1$ rounds on the critical path, and the trace's own `rounds` field reports 1 at every size, which is a disagreement about what a round is rather than a defect in the algorithm — but it is a disagreement that hides the only cost this algorithm has. For agent ranks the important property is the one that distinguishes a scatter from posting the task list to the group chat: rank d receives `payloads[d]` and nothing else, so no rank pays context for work assigned to somebody else. Every one of the 31 receives at $p=32$ is logged `admitted: false` with `admitted_tokens: 0` and every rank finalises with a context high-water of 0, which is the admission half of that guarantee working.

1 What this family is

The `scatter` collective under the `linear` algorithm, measured across 21 runs at 13 distinct process counts from 2 to 32. Every run is agent-free and deterministic, so the measurements are of the protocol itself.

2 What the analysis notices

- [ok] All 21 measured sizes logged exactly the number of messages the closed form predicts.
- [note] Wall times span 0.012–0.078 s with no agent in the loop, so these measure the fabric and the protocol rather than model latency.

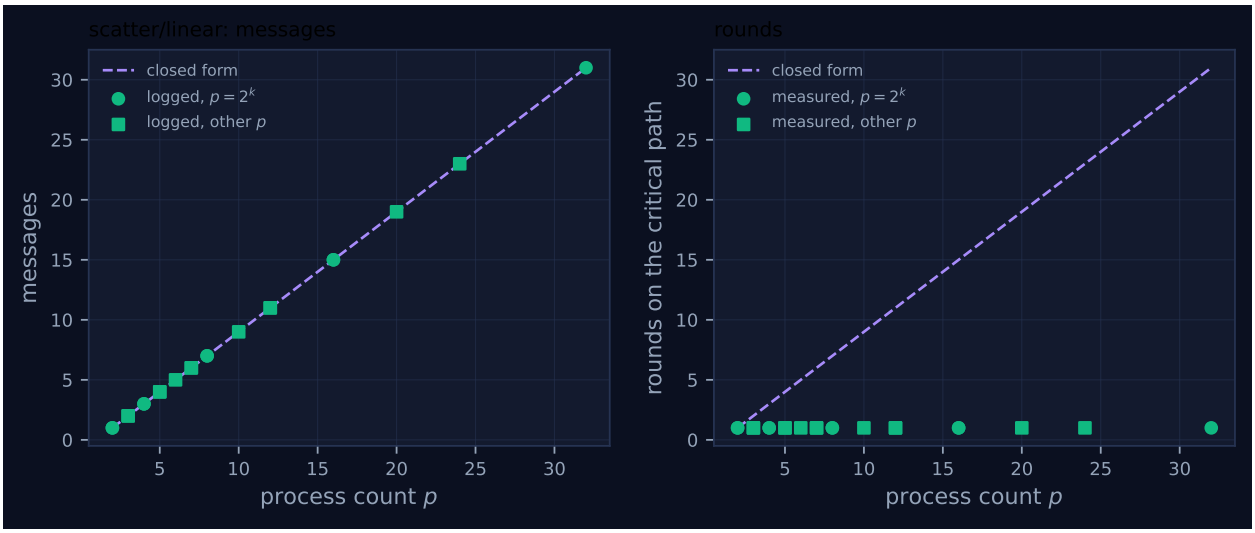

Figure 1: Measured message count and critical-path rounds against process count, with the closed-form prediction. Powers of two are drawn as circles and other sizes as squares, because algorithms take a different code path at sizes that are not powers of two. A red cross marks a size where the collective's own reported count disagrees with the traffic the fabric logged.

3 Every measured size

p	campaign	logged	pred.	self-rep.	rounds	pred.	wall (s)	model	acct.
2	`mb-free`	1	1	1	1	1	0.012	✓	✓
2	`mb-smoke`	1	1	1	1	1	0.012	✓	✓
3	`mb-free`	2	2	2	1	2	0.013	✓	✓
3	`mb-smoke`	2	2	2	1	2	0.014	✓	✓
4	`mb-free`	3	3	3	1	3	0.016	✓	✓
4	`mb-smoke`	3	3	3	1	3	0.014	✓	✓
5	`mb-free`	4	4	4	1	4	0.015	✓	✓
5	`mb-smoke`	4	4	4	1	4	0.015	✓	✓
6	`mb-free`	5	5	5	1	5	0.030	✓	✓
7	`mb-free`	6	6	6	1	6	0.020	✓	✓
7	`mb-smoke`	6	6	6	1	6	0.032	✓	✓
8	`mb-free`	7	7	7	1	7	0.023	✓	✓
8	`mb-smoke`	7	7	7	1	7	0.023	✓	✓
10	`mb-free`	9	9	9	1	9	0.025	✓	✓
12	`mb-free`	11	11	11	1	11	0.035	✓	✓
12	`mb-smoke`	11	11	11	1	11	0.027	✓	✓
16	`mb-free`	15	15	15	1	15	0.039	✓	✓
16	`mb-smoke`	15	15	15	1	15	0.043	✓	✓
20	`mb-free`	19	19	19	1	19	0.057	✓	✓
24	`mb-free`	23	23	23	1	23	0.069	✓	✓
32	`mb-free`	31	31	31	1	31	0.078	✓	✓

4 How the algorithm scales

The implementation is a loop and a receive. The root iterates `for d in range(p)`, skips itself, and sends `payloads[d]` to rank d; every other rank performs a single `_crecv` from the root. So the message count is $p - 1$, the volume $(p - 1)n$, and the fold depth zero — which is

`FORMULAS["scatter","linear"]`, whose round term is $p - 1$.

That round term is the number to understand, because it is the entire difference between this algorithm and its binomial sibling; the message counts are identical. The root's $p - 1$ sends are issued from one thread of control, one after another, so $p - 1$ message costs lie in sequence on the path from the collective's start to the last rank receiving its work. The measurements make this concrete in a way the counts cannot: at $p = 32$ the fabric log contains 31 `msg.send` records and *every one of them is from rank 0.* No other rank sends anything. Each of the thirty-one other ranks performs exactly one receive.

The message panel of Figure 1 is therefore the least interesting plot in the sweep — the line $p - 1$, matched at all twenty-one rows, 1 at $p = 2$ through 31 at $p = 32$, with the self-reported volume also $p - 1$ because the benchmark's `f"u{i}"` work units tokenise to one token each. The round panel is where the family view earns its keep, and what it shows is a measured 1 against a predicted $p - 1$ at every size from $p = 3$ upward.

That gap is definitional. The implementation writes `tr.stats.rounds = 1`, meaning one logical phase in which all the sends happen. The closed form writes $p - 1$, meaning the root's serialised path. The module is inconsistent about which it records, and the inconsistency is visible within a few hundred lines: `bcast/flat`, which is structurally the same fan-out with the same payload going to every destination, records `1 if r != root else p - 1` — the serialised reading, made rank-dependent, which is the honest choice and the one that would have made this column agree. So the `rounds` column here should be read as "phases the algorithm believes it executed", and the $p - 1$ beside it as the quantity a harness author actually needs.

The log supports the serialised reading. Aggregate receive-blocking at $p = 32$ is 0.32 rank-seconds spread across the thirty-one waiting ranks, and it grows as $p^{1.35}$ ($R^2 = 0.956$): each rank waits for the root to reach it, so the mean wait grows with p and the total grows a little faster than linearly. The viewer screenshot for `mb-free__coll-scatter-linear-32` shows one lane with a run of activity spanning the full 0.08 s and thirty-one lanes with a single tick each, arriving progressively later down the distribution order. That is a picture of a queue at a single server, which is what a linear scatter is.

5 Powers of two and everything else

There is nothing power-of-two about this algorithm: the send loop is `range(p)` with a skip, the receive names the root explicitly, and no bit arithmetic appears anywhere in the branch. The family view confirms the expected consequence — the squares in Figure 1 lie on $p - 1$ exactly as the circles do, 2, 4, 5, 6, 9, 11, 19 and 23 messages at $p = 3, 5, 6, 7, 10, 12, 20, 24$ — with no red crosses, meaning the collective's self-reported count equalled the fabric's logged traffic at all twenty-one sizes. The eight sizes measured in both `mb-free` and `mb-smoke` agree on every integer count and differ only in wall time, for instance 0.023 s against 0.023 s at $p = 8$ and 0.039 s against 0.043 s at $p = 16$.

That makes this family the control for its op, in the same way `gather/linear` is for gather. `scatter/binomial` does have a remainder path — its subtrees are truncated by p — and its volume closed form is inexact at non-powers of two as a result. Establishing that a sibling with no bit arithmetic agrees with its closed form at all twenty-one sizes rules out the measurement apparatus as the source of that inexactness.

One difference between the two algorithms is not a cost and does not appear in the sweep, but it hides at exactly the size boundary this family covers. `scatter()` picks its algorithm as `"linear"`

`if p <= 4 else "binomial"` when the caller does not name one. The `linear` branch passes the caller's `admit` argument through to the receive; the `binomial` branch hard-codes `admit=False` and ignores the argument. So `comm.scatter(units, admit=True)` places a rank's work unit in its context at $p = 4$ and silently does not at $p = 5$. All twenty-one runs here leave `admit` at its default and so log `admitted: false` throughout; the finding is from reading the two branches together, which is what a family document can do and a single run cannot.

6 When to choose this algorithm

`scatter` and `gather` are duals, and within each op the choice between `linear` and `binomial` is the same choice: depth against volume at equal message count. Both algorithms move $p - 1$ messages at every one of the twenty-one sizes in both families, so nothing is traded in traffic. `linear` wins on volume by exactly the closed forms' ratio, $(p - 1)n$ against $np\lceil\log_2 p\rceil/2$, which at $p = 32$ is 31 units against 80 — both confirmed by the collectives' self-reports. It loses on depth by $p - 1$ against $\lceil\log_2 p\rceil$, 31 against 5. `cost.predict` at $n = 4{,}096$ tokens per unit gives 15.97 s of critical path against 2.58 s and 126,976 tokens against 327,680, so `cost.best_algorithm` answers `linear` on the volume objective and `binomial` on the time objective at every (p, n) pair I evaluated. The two objectives disagree honestly.

The tie-breaker for agent ranks is that the root is not replicable, and here that argument is starker than for gather. Under `linear` the root issues every message itself: at $p = 32$, 31 of 31 sends came from rank 0 and no other rank sent anything. If the root is an agent, work distribution is a serial section of length $p - 1$ that does not shrink as the population grows — the textbook Amdahl term, and in an agent harness the phase that dominates f. Under `binomial` the root issues 5 sends at $p = 32$ and 15 other ranks share the forwarding. `linear` is the right choice when p is small, when the work units are large enough that carrying them through interior ranks is the dominant cost, or when the harness needs the property that a unit passes through exactly one hop.

That last property is the one worth stating carefully, because it is what separates a scatter from the obvious alternative. Posting the whole task list to a group chat costs every rank the full p-way list in context; a scatter costs rank d only `payloads[d]`. Under `linear` that is exact — one message, one destination, one unit — and the sweep shows the admission side of it holding: at $p = 32$ all 31 receives are logged `admitted: false` with `admitted_tokens: 0`, and all thirty-two ranks finalise with a context high-water of 0 against a 128,000-token budget. Under `binomial` the same guarantee holds for a different reason: interior ranks forward blocks belonging to their subtree, so they see other ranks' assignments, and the branch receives them with `admit=False` so that no interior agent reads them. Both algorithms deliver work by handle rather than by value; `linear` does it without ever showing a unit to a rank that does not own it.

7 Threats to this reading

No agents took part, so nothing here concerns model behaviour. No executor is recorded, the viewer reports zero agent calls, and the wall times of 0.012–0.078 s are SQLite writes and event-log appends. The serialisation measurements — 31 of 31 sends from one rank, 0.32 rank-seconds of aggregate waiting, the $p^{1.35}$ fit — are fabric behaviour under thread contention. They establish the *shape* of the fan-out, which is what the algorithm determines, and say nothing about the cost of distributing work to models.

The work units are one token each, which suppresses the effect that matters. The benchmark scatters `f"u{i}"`, so the whole collective moves 31 tokens at $p = 32$ and no context budget or transport threshold could be reached. Every claim above about context cost is an argument from the closed form and from the real runs, not a measurement from this family. In `real-sw-p8-full` the same op with realistic payloads shows what changes: within a single scatter, `Mode.AUTO` sent a 2,077-token block by `rendezvous` and 1,044- and 527-token blocks by `eager`, splitting at the 2,048-token eager limit. Here that mechanism was never engaged.

The transport mode is a launch artefact. All messages are eager because the benchmark launched with `eager_limit=10**9`, which forces `Mode.AUTO` to eager for any payload, and with `strict_context=False`, which disables admission enforcement. The `admitted: false` records remain meaningful as a code-path property — the collective passes `admit=False` into the receive — but a reader should not infer that `Mode.AUTO` would have chosen eager for a realistic unit, because it was never offered the choice.

The round-count gap is definitional and I have not established which convention is intended. The implementation says 1, the closed form says $p - 1$, and `bcast/flat` in the same module says $p - 1$ at the root for the mirror-image operation. My reading is that the serialised convention is the useful one; someone who intended `rounds` to count phases would say the closed form needs changing instead. Either way they disagree today, and no tooling compares them: `bench_collectives` records both `rounds_measured` and `rounds_model` and then tests only `messages_match` and `fold_depth_match`.

The `admit` asymmetry is read from source, not observed. No run in this sweep passes `admit=True`, so the claim that the binomial branch ignores it rests on the literal `False` in its call against the pass-through in this one. It is a short reading but not a measurement.